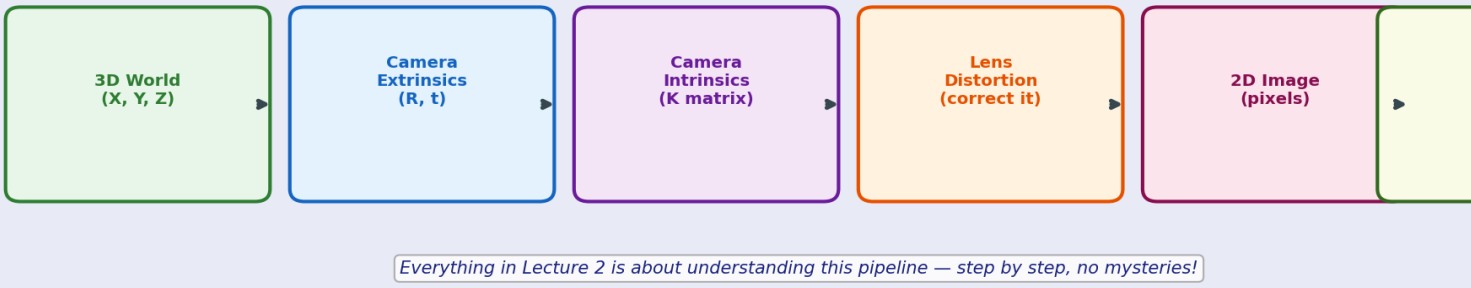
Computer Vision

Lecture 2 — Image Formation & Camera Models

A Beginner-Friendly, Super-Detailed Study Guide

Prepared for: Tomorrow's Class | April 2026

Fig 8 — Lecture 2 Big Picture: From 3D World to Image



Everything in Lecture 2 is about understanding this pipeline — step by step, no mysteries!

Topics Covered: Pinhole Camera | Projection Math | Intrinsic & Extrinsic Params | Distortion | Calibration | Pixels

0. Quick Recap — Where We Left Off

In Lecture 1 we answered the question: "What IS computer vision?" — we saw that CV is about making computers understand images, and we looked at the big picture: the AI family tree, the Vision System (Geometry, Physics, Learning Theory), and all the cool applications.

Now Lecture 2 zooms in on the very FIRST step of any vision system:

> **Core Question** Before we can detect faces, reconstruct 3D scenes, or do anything fancy — we need to understand: How does a camera turn the 3D real world into a flat 2D image? That is the entire point of this lecture.

Think of it this way: your eye does this automatically. But a computer needs us to write down the exact math. Once we have that math, we can reverse it — and go from 2D images back to 3D, which is what makes AR, robotics, and 3D reconstruction possible.

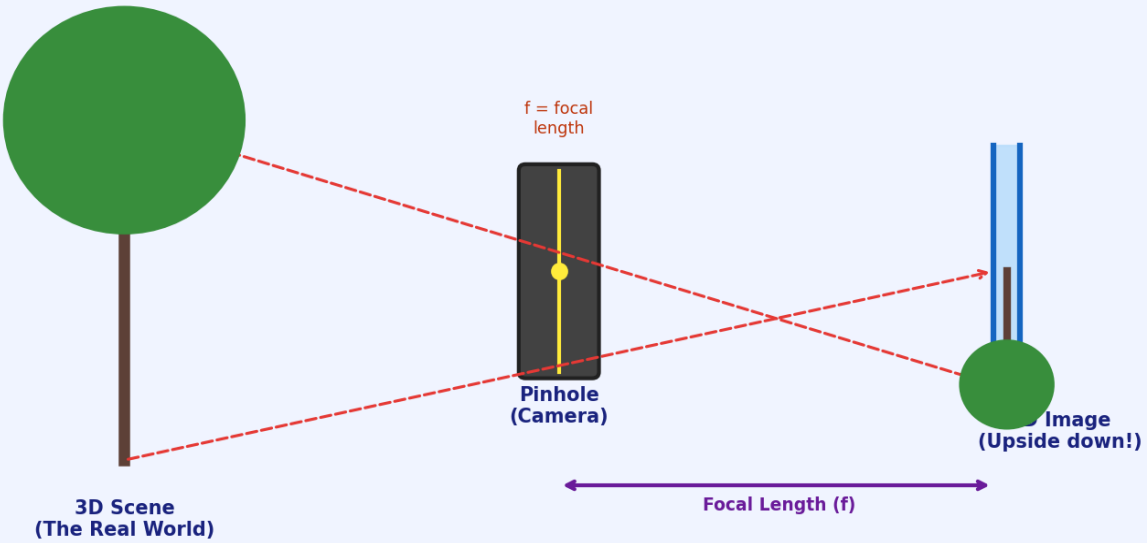
1. The Pinhole Camera Model

1.1 What Is a Pinhole Camera?

Imagine a completely dark box. You poke a tiny hole (the "pinhole") in one wall. Light from the outside world enters through the hole and falls on the opposite wall — creating an upside-down image of the world. That is it! That is a pinhole camera.

Your phone camera, DSLR, and even your eye all work on this same basic principle — just with a lens to gather more light instead of a tiny hole.

Fig 1 — How a Pinhole Camera Works



KEY INSIGHT: The image is upside-down because rays from the top of the scene pass through the hole and hit the BOTTOM of the image plane. Rays from the bottom hit the TOP. This is normal and expected — software just flips it back.

1.2 The Most Important Parts

Term	What It Means	Simple Analogy

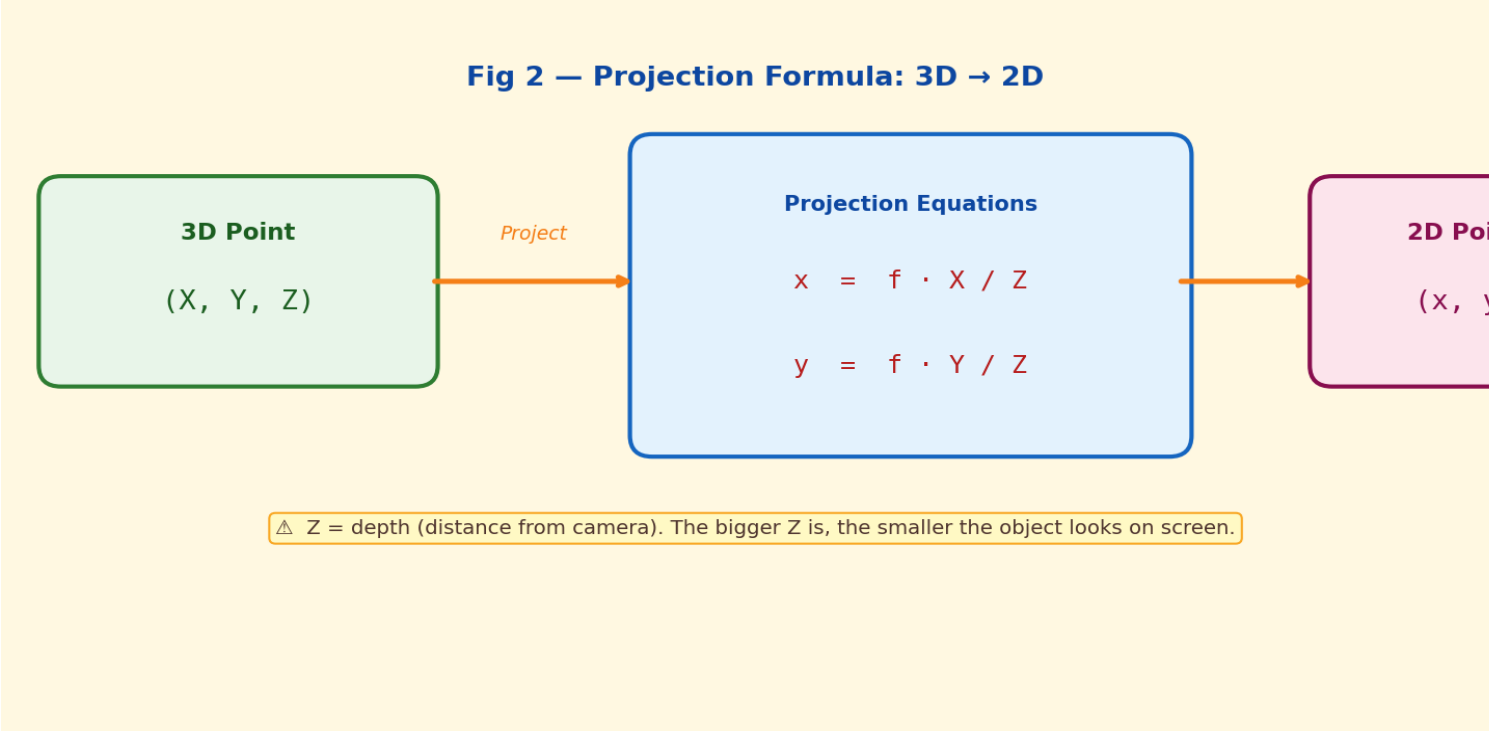
Focal Length (f)	Distance from pinhole to the image plane. Controls how zoomed-in the image looks.	Zoom level on your phone camera
Image Plane	The flat surface where the image forms (like film in old cameras, or the sensor chip in digital ones)	The back wall of the dark box
Principal Point (cx, cy)	The exact centre of the image — where the main axis hits the image plane	The bull's-eye target in the middle of the image
Field of View (FOV)	How wide an angle the camera can see	Wide-angle vs telephoto lens
Optical Axis	An imaginary line going straight through the pinhole, perpendicular to image plane	The camera's "looking direction"

2. Projection Math — How 3D Becomes 2D

2.1 The Simple Idea

We have a 3D point in the world at coordinates (X, Y, Z). We want to know where it ends up on our 2D image — i.e., what pixel (x, y) does it map to?

The answer comes from simple similar-triangles geometry (you probably learned this in high school!):



2.2 Understanding the Formulas

x = f · X / Z means: the horizontal position on the image equals focal-length times (the horizontal distance in the world) divided by (how far away it is).

y = f · Y / Z same idea for the vertical direction.

Breaking this down with a real example:

- Imagine a ball at X=2m, Y=0m, Z=10m away (straight ahead, slightly to the right)
- Camera has focal length f = 500 (in pixel units)
- x = 500 × 2 / 10 = 100 pixels to the right of centre
- y = 500 × 0 / 10 = 0 (at the vertical centre)

Why things look smaller far away The "divide by Z" part is why things look SMALLER when they are FAR AWAY. If Z doubles (object moves twice as far), x halves (looks half the size). This is perspective! Your brain does this automatically.

2.3 The Full Projection Matrix

In real life we express all this using matrices (a table of numbers). A matrix lets us do the whole projection in one single step using matrix multiplication. The full formula looks like:

$$\begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} f & 0 & cx \\ 0 & f & cy \\ 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} X \\ Y \\ Z \end{bmatrix} \quad \text{then divide everything by } Z$$

That 3×3 matrix in the middle is called the K matrix or Intrinsic Matrix. It stores everything about the internal properties of your camera.

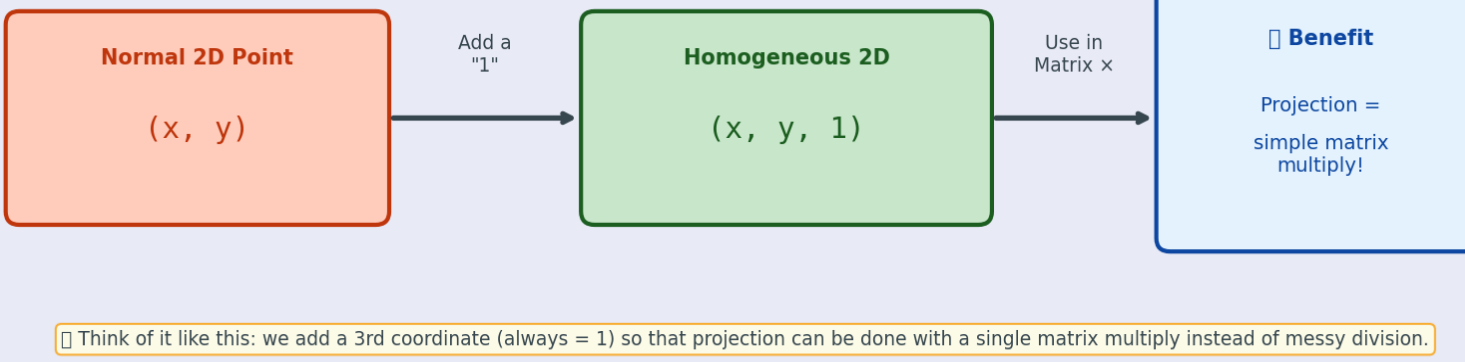
Do not worry if matrix math looks scary. The key point is: one single matrix (K) captures ALL the important info about your camera's internal properties. You need this to go from pixel coordinates back to real-world measurements.

3. Homogeneous Coordinates — A Clever Math Trick

3.1 The Problem

The projection formula involves dividing by Z, which is annoying — division makes equations messy. Mathematicians invented a trick called "homogeneous coordinates" to turn this division into a simple matrix multiply.

Fig 4 — Homogeneous Coordinates: The Magic Trick



Think of it like this: we add a 3rd coordinate (always = 1) so that projection can be done with a single matrix multiply instead of messy division.

3.2 How It Works

Instead of writing a 2D point as (x, y), you write it as (x, y, 1) — just add a "1" at the end.

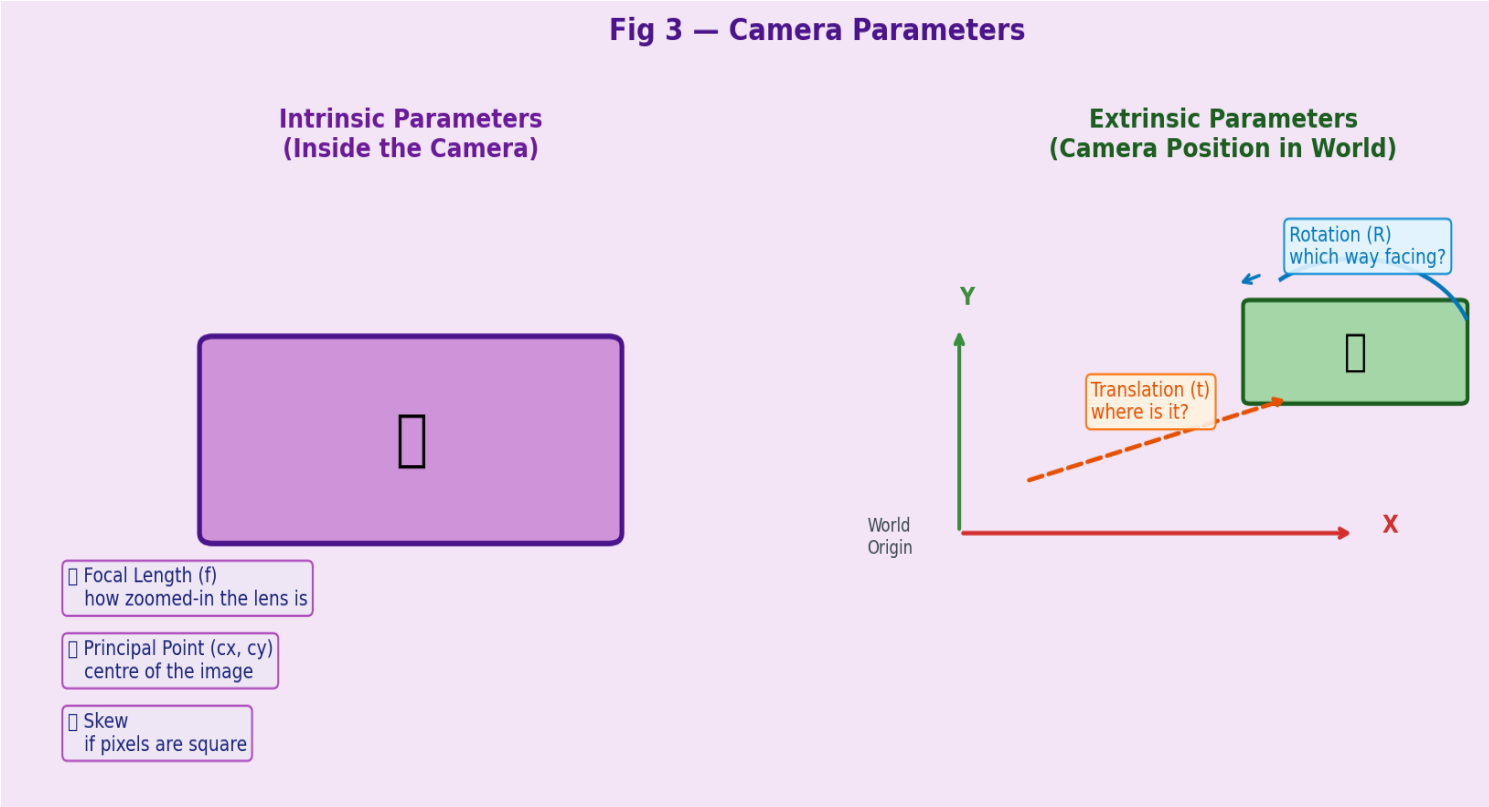
Instead of writing a 3D point as (X, Y, Z), you write it as (X, Y, Z, 1).

Now the whole projection — including the "divide by Z" part — can be written as a single matrix multiplication. This is way cleaner to work with in code.

Rule of thumb You will see homogeneous coordinates EVERYWHERE in CV. Whenever you see a 2D point written as (x, y, 1) or a 3D point as (X, Y, Z, 1) with a mysterious extra number — that is homogeneous coordinates. It is just a math convenience, nothing magical.

4. Intrinsic vs Extrinsic Parameters

Every camera has two types of parameters that fully describe it. Think of them as "who the camera IS" versus "WHERE the camera is."



4.1 Intrinsic Parameters (Inside the Camera)

These describe the internal properties of the camera itself. They stay the same no matter where you place the camera.

Parameter	Symbol	What It Is	Typical Value
Focal Length	f	How zoomed in the lens is. Larger = more zoomed.	500–2000 pixels
Principal Point x	cx	Horizontal centre of the image	Width / 2 (e.g. 640)
Principal Point y	cy	Vertical centre of the image	Height / 2 (e.g. 360)
Skew	s	Whether pixels are perfectly rectangular. Usually 0.	0 (almost always)

All four numbers are stored together in the K matrix (also called Camera Matrix or Intrinsic Matrix):

```
K  =  [ [f, s, cx],  [0, f, cy],  [0, 0, 1] ]
```

The K matrix is the "fingerprint" of your camera. Every camera model has a different K. Once you calibrate your camera (find its K), you can use it forever — until the lens changes.

4.2 Extrinsic Parameters (Camera in the World)

These describe WHERE the camera is placed and WHICH WAY it is pointing. They change every time you move the camera.

- Rotation Matrix (R): a 3×3 matrix describing which direction the camera is facing. Think of it as: "has the camera been tilted, rotated, or turned?"
- Translation Vector (t): a 3×1 vector describing where the camera is in 3D space. Think of it as: "how far has the camera moved from the origin (0,0,0)?"

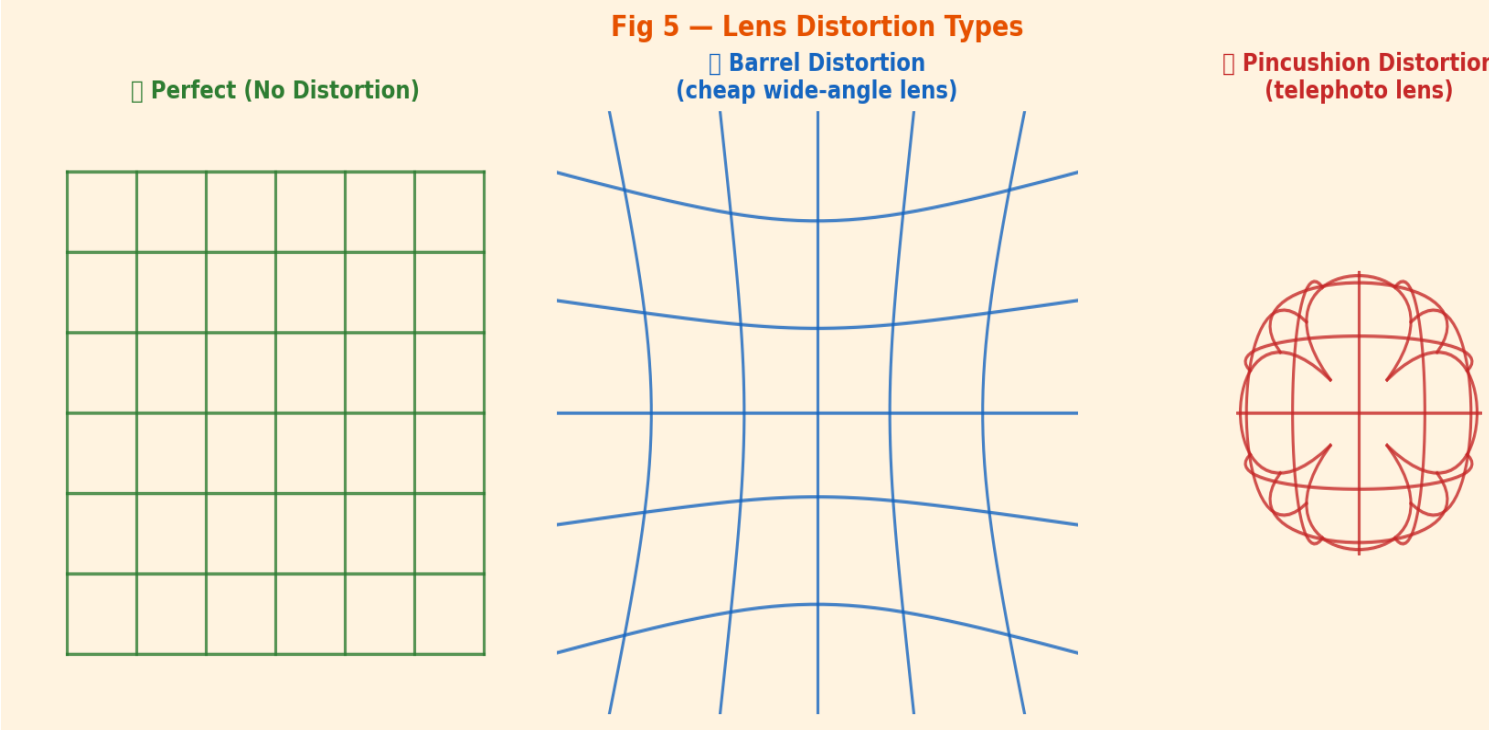
Together, R and t tell you the "pose" of the camera — its complete position and orientation in the world. This is crucial for 3D reconstruction, robotics, and AR.

Easy Analogy Analogy: Intrinsics = properties of your phone camera (resolution, zoom). Extrinsics = where you are standing and which direction you are pointing your phone. Same phone, different extrinsics every time you move.

5. Lens Distortion

5.1 Why Do We Get Distortion?

A perfect pinhole camera has no lens — light passes through a single point. But real cameras need a LENS to gather enough light (a real pinhole is too dark). Lenses are curved pieces of glass, and curved glass bends light in ways that cause distortion.



5.2 Types of Distortion

Type	What It Looks Like	Cause	Common In
Barrel Distortion	Straight lines bow OUTWARD (like a barrel). Centre looks bigger.	Short focal length, cheap wide-angle lenses	GoPro cameras, phone cameras, fisheye lenses
Pincushion Distortion	Straight lines bow INWARD (like a pincushion). Edges look stretched.	Long focal length, telephoto lenses	Zoom lenses, telescope cameras
Tangential Distortion	Image looks tilted or shifted unevenly	Lens not perfectly parallel to sensor	Cheap cameras, manufacturing defects

5.3 The Distortion Equation

Distortion is described by coefficients: k1, k2 (radial) and p1, p2 (tangential). The correction formula is:

$$x\_corrected = x(1 + k1 \cdot r^2 + k2 \cdot r^4) + 2p1 \cdot xy + p2(r^2 + 2x^2)$$

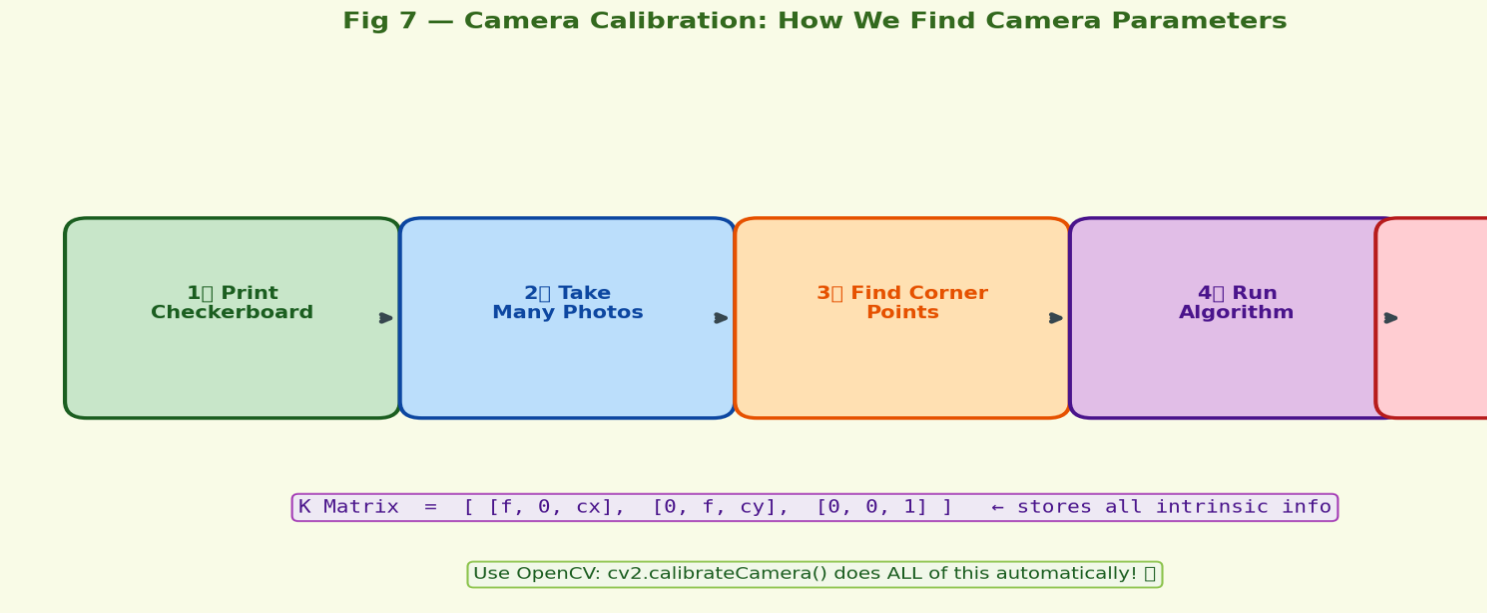
where $r^2 = x^2 + y^2$ is the distance from the image centre.

IMPORTANT: You do NOT need to do this by hand. OpenCV's cv2.undistort() does this automatically once you know the distortion coefficients from calibration. Just understand what it is doing conceptually.

6. Camera Calibration

6.1 What Is Calibration?

Camera calibration is the process of figuring out all the intrinsic parameters (focal length, principal point) AND the distortion coefficients of a specific camera. Think of it as "learning the camera's math fingerprint."



6.2 Step-by-Step Calibration Process

Here is exactly how it works in practice:

1. **Print a checkerboard pattern.** The checkerboard has a known size — we know exactly how many squares and how big each square is in real-world centimetres.
2. **Take 10–20 photos of the checkerboard.** Hold it at different angles, distances, and positions. The more variety, the better the calibration.
3. **OpenCV finds the corners automatically.** The function cv2.findChessboardCorners() detects the exact pixel coordinates of every corner in every photo.
4. **Run cv2.calibrateCamera().** It uses the known real-world positions of the corners AND their pixel positions to solve for K, R, t, and distortion coefficients mathematically.
5. **You get back the K matrix and distortion coefficients!** Save these — you can now use them to correct any image from this camera, or to do accurate 3D measurements.

Good News The math behind calibration (Zhang's method, 1999) is advanced. But using it in OpenCV is just 5 lines of code. Your professor may show you the code in the next lab — it is surprisingly easy to use even if the internals are complex.

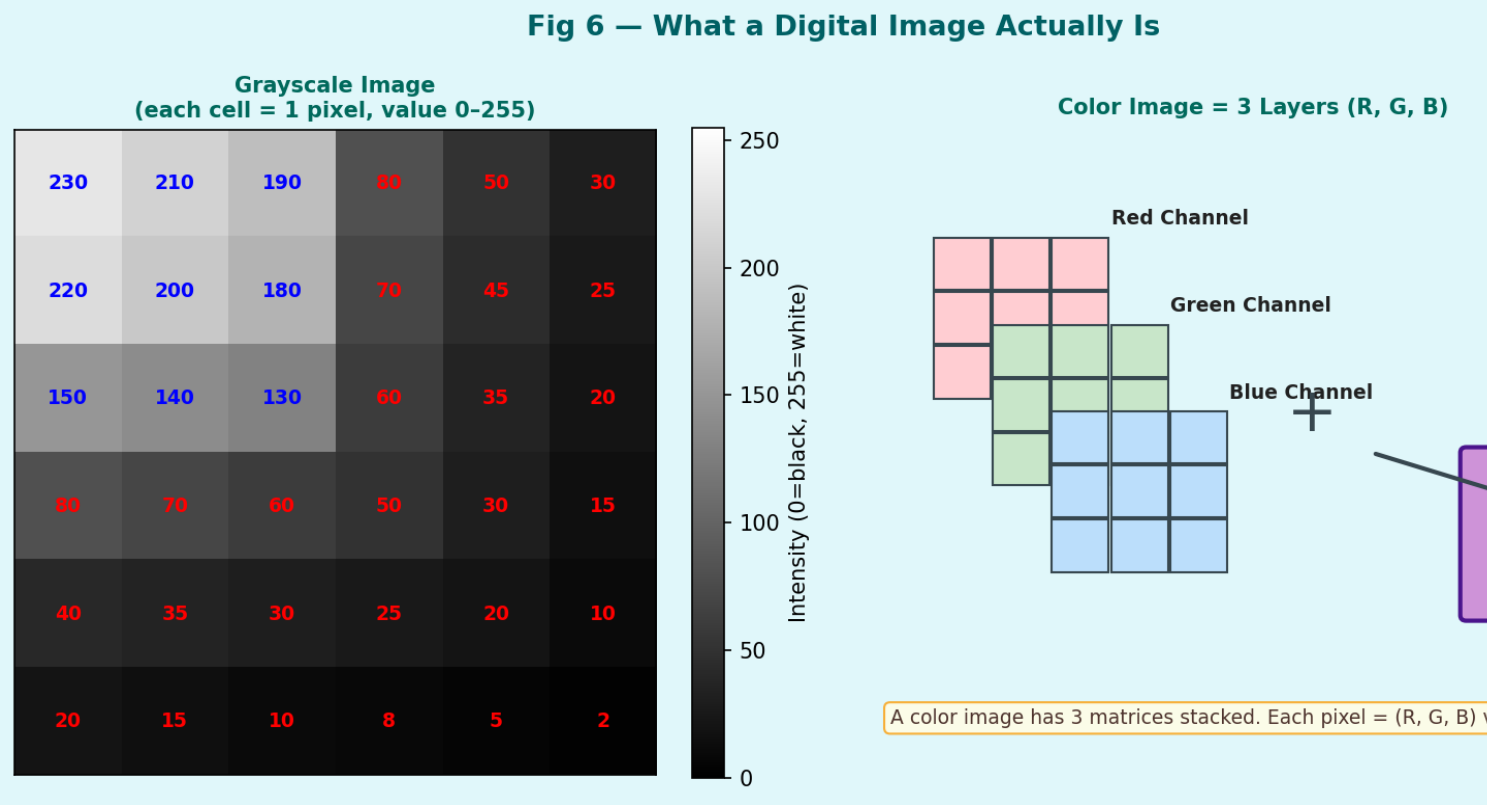
6.3 Why Does Calibration Matter?

- Without calibration, you cannot accurately measure real-world distances from images
- Without calibration, distortion makes straight lines look curved in your CV output
- For robotics, AR, and 3D reconstruction — calibration is non-negotiable
- Modern smartphones do auto-calibration inside the camera firmware, but for research or precision work, you must calibrate manually

7. What Is a Digital Image? (Image as a Matrix)

7.1 Pixels and Intensity Values

Once the camera projects the 3D world onto the image plane, it captures that image as a grid of tiny squares called pixels. Each pixel stores a number representing how bright or what colour that tiny area is.



7.2 Grayscale Images

A grayscale image is a 2D array (matrix) of numbers. Each number is between 0 and 255:

- 0 = completely black
- 128 = medium grey
- 255 = completely white

A 1920×1080 grayscale image is simply a matrix with 1080 rows and 1920 columns, containing 2,073,600 numbers.

7.3 Colour (RGB) Images

A colour image has THREE such matrices stacked together — one for Red, one for Green, and one for Blue. Each pixel is stored as three numbers: (R, G, B).

- (255, 0, 0) = pure red
- (0, 255, 0) = pure green
- (0, 0, 255) = pure blue
- (255, 255, 0) = yellow (red + green)
- (255, 255, 255) = white (all channels max)
- (0, 0, 0) = black (all channels zero)

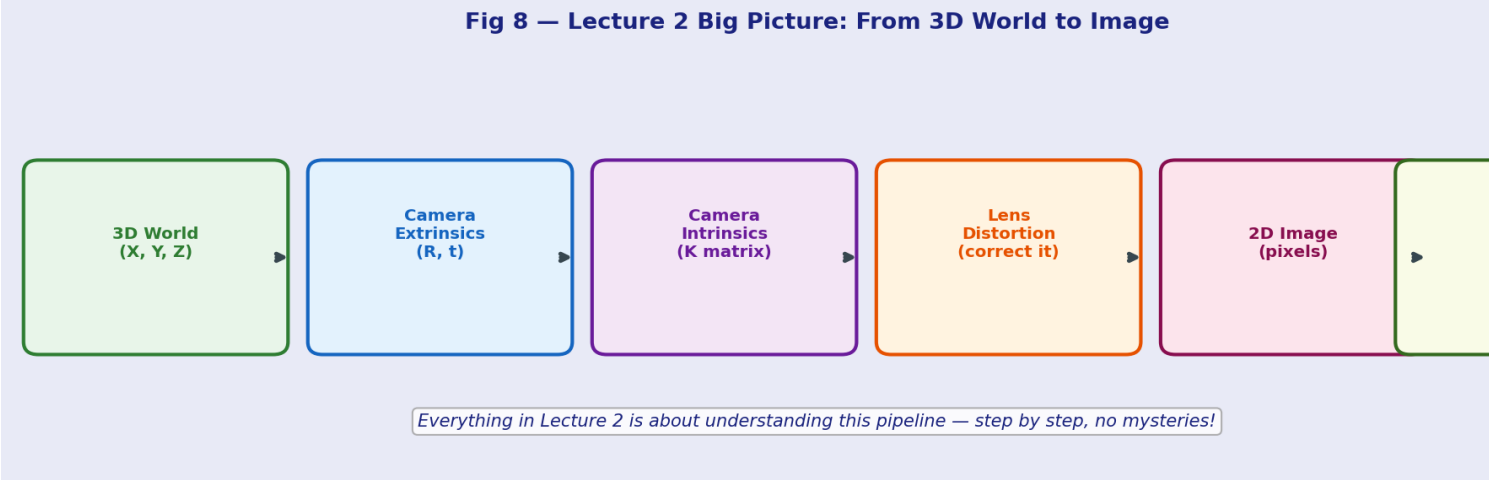
So a 1920×1080 colour image is a 3D array with shape (1080, 1920, 3) — containing 6,220,800 numbers total.

> OpenCV IMPORTANT: OpenCV stores colour images as BGR (Blue-Green-Red) NOT RGB! This is a historical quirk. Always be careful when converting between OpenCV and other libraries like matplotlib or PIL which use RGB.

7.4 Data Types

Type	Value Range	Common Use
uint8 (8-bit)	0 to 255	Standard photos, webcam frames, most images
float32	0.0 to 1.0 (normalized)	Neural network input, after dividing by 255
float64	0.0 to 1.0	High-precision scientific imaging
uint16	0 to 65535	Medical imaging (CT, MRI), depth maps

8. Putting It All Together — The Complete Pipeline



Here is how the whole Lecture 2 story connects:

Step	What Happens	Key Concept
1	3D world exists with objects at real coordinates (X, Y, Z)	3D Euclidean Space
2	Extrinsic parameters (R, t) define WHERE the camera is relative to the world	Camera Pose
3	The K matrix (intrinsics) maps from camera coordinates to image coordinates using the projection formula x = f*X/Z	Perspective Projection
4	Real lens causes distortion — barrel or pincushion — described by distortion coefficients	Lens Distortion
5	Sensor captures light and stores it as a grid of pixel values (0-255) per channel	Digital Sampling
6	Result: a numpy array you can load in Python with cv2.imread()	Digital Image Array

The Big Payoff Calibration lets you work backwards: given a pixel position (x, y), you can compute the real-world ray in 3D space that created it. Combined with depth info (from stereo cameras or LiDAR), you can reconstruct the 3D world. That is the foundation of 3D reconstruction, AR, and robotics!

9. OpenCV in Practice — What You'll Actually Code

9.1 Reading and Displaying an Image

```
import cv2
import numpy as np

# Load image (stored as a numpy array)
img = cv2.imread("photo.jpg")  # shape: (H, W, 3) - BGR
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)  # to grayscale
print(img.shape)   # e.g. (720, 1280, 3)
print(img.dtype)   # uint8
cv2.imshow("My Image", img); cv2.waitKey(0)
```

9.2 Camera Calibration in Code

```
# --- CALIBRATION (run once per camera) ---
import glob
objpoints = []  # 3D real-world corner coords
imgpoints = []  # 2D pixel coords found in image

for fname in glob.glob("calib_images/*.jpg"):
    img = cv2.imread(fname)
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    ret, corners = cv2.findChessboardCorners(gray, (9, 6))
    if ret: imgpoints.append(corners)

ret, K, dist, rvecs, tvecs = cv2.calibrateCamera(...)
# K is your intrinsic matrix, dist is distortion coefficients
```

9.3 Undistorting an Image

```
# --- UNDISTORTION ---
undistorted = cv2.undistort(img, K, dist)
# That's it! Straight lines are now straight again.
```

IMPORTANT: Always remember: OpenCV uses BGR, not RGB. When you do cv2.imread() the channels are Blue, Green, Red — not Red, Green, Blue. Use cv2.cvtColor(img, cv2.COLOR_BGR2RGB) to convert for matplotlib.

10. Questions Your Professor Might Ask

Question	Short Answer
What is the pinhole camera model?	A simplified camera model where all light passes through a single point (the pinhole). 3D points project to 2D using x=fX/Z, y=fY/Z.
What are intrinsic parameters?	Properties INSIDE the camera: focal length (f), principal point (cx, cy), skew. Stored in the K matrix.
What are extrinsic parameters?	The camera's position and orientation in the world: Rotation matrix R and translation vector t.
Why do we use homogeneous coordinates?	To express the projection equation as a clean matrix multiplication instead of messy division by Z.
What is barrel distortion?	A type of lens distortion where straight lines bow outward (away from centre). Common in wide-angle cameras.
What is camera calibration?	The process of finding a camera's K matrix and distortion coefficients, usually using a checkerboard pattern.
How is a digital image stored in memory?	As a numpy array with shape (Height, Width, Channels). Grayscale: 2D array. Colour: 3D array with 3 channels (BGR in OpenCV).
What is the K matrix?	The 3x3 intrinsic matrix [[f,0,cx],[0,f,cy],[0,0,1]] that converts camera-space 3D coords to image pixel coords.

11. One-Page Cheat Sheet

Concept	Formula / Key Info
Projection (basic)	$x = f \cdot X/Z, \quad y = f \cdot Y/Z$
K matrix (intrinsics)	[[f, 0, cx], [0, f, cy], [0, 0, 1]]
Full projection	pixel = K · [R | t] · world_point (in homogeneous coords)
Radial distortion correction	$x' = x(1 + k1 \cdot r^2 + k2 \cdot r^4)$
Image shape in OpenCV	(Height, Width, 3) — NOT (Width, Height)!
Pixel value range	0 (black) to 255 (white) for uint8
OpenCV colour order	BGR — not RGB!
Calibration function	cv2.calibrateCamera(objpoints, imgpoints, imageSize, None, None)
Undistortion function	cv2.undistort(img, K, dist_coeffs)
Find checkerboard corners	cv2.findChessboardCorners(gray, (cols-1, rows-1))

Pre-Lecture Prep Before the lecture: review basic matrix multiplication (dot product of rows and columns). Even if you do not fully remember it, just knowing that matrices are "tables of numbers used to transform coordinates" is enough to follow along.

12. Glossary of New Terms

Term	Definition
Pinhole Camera	A simplified camera model with a single point opening that projects 3D scenes to 2D
Focal Length (f)	Distance from the pinhole/lens to the image sensor. Controls zoom level.
Image Plane	The flat surface (sensor) where light falls to form the image
Projection	The mathematical process of mapping 3D points to 2D image coordinates
Intrinsic Matrix (K)	3x3 matrix containing focal length, principal point — fixed properties of the camera
Extrinsic Parameters	Rotation (R) and Translation (t) — how the camera is positioned in the world
Homogeneous Coordinates	A coordinate system that adds an extra "1" to allow projection as matrix multiplication
Barrel Distortion	Lens distortion where straight lines bow outward, common in wide-angle lenses
Pincushion Distortion	Lens distortion where straight lines bow inward, common in telephoto lenses
Camera Calibration	Process of finding a camera's K matrix and distortion coefficients
Distortion Coefficients	Numbers (k1, k2, p1, p2) that describe how much and what type of distortion a lens has
Pixel	The smallest unit of a digital image. Stores intensity value(s) 0-255.
BGR	OpenCV's channel order: Blue, Green, Red (opposite of the standard RGB)
uint8	Data type for images: unsigned 8-bit integer, range 0-255
Principal Point	The point in the image that the optical axis passes through (approximately image centre)
Field of View	The angle of the scene that a camera can see. Wider angle = shorter focal length.

Summary — What You Learned Today

Lecture 2 gave you the mathematical foundation for ALL of computer vision. Here is the one-paragraph summary:

A camera projects the 3D world onto a 2D image plane using the pinhole camera model. The projection math ($x = f \cdot X/Z$) is captured neatly in the K matrix (intrinsics). The camera's position and orientation in the world are described by R and t (extrinsics). Real lenses cause distortion (barrel or pincushion) which we model with distortion coefficients. Camera calibration is the process of finding K and the distortion coefficients from checkerboard images. The final result — the digital image — is stored as a matrix of pixel values (0–255), available in Python as a NumPy array via cv2.imread().

NEXT STEPS: Lecture 3 will most likely cover Image Processing Basics — how to filter, blur, sharpen, and detect edges in images. The pinhole model and pixel representation from today are the prerequisites for that. Good luck tomorrow!